```python
import qutip as qt
import numpy as np

# Matrice identité 2x2 et de Pauli
I = qt.qeye(2)
Sx = qt.sigmax()
Sy = qt.sigmay()
Sz = qt.sigmaz()

# produit tensoriel pour trois particules de spin 1/2

Sx1 = qt.tensor(Sx, I, I)
Sy1 = qt.tensor(Sy, I, I)
Sz1 = qt.tensor(Sz, I, I)

Sx2 = qt.tensor(I, Sx, I)
Sy2 = qt.tensor(I, Sy, I)
Sz2 = qt.tensor(I, Sz, I)

Sx3 = qt.tensor(I, I, Sx)
Sy3 = qt.tensor(I, I, Sy)
Sz3 = qt.tensor(I, I, Sz)

# calcul des multiplications des matrice de Pauli

S1_S2 = Sx1*Sx2 + Sy1*Sy2 + Sz1*Sz2
S1_S3 = Sx1*Sx3 + Sy1*Sy3 + Sz1*Sz3
S2_S3 = Sx2*Sx3 + Sy2*Sy3 + Sz2*Sz3

# construction de l'hamiltonien

H = -(S1_S2 + S1_S3 + S2_S3)

# calcul des valeurs propres de l'hamiltonien

valeurs_propres, vecteurs_propres = H.eigenstates()

# Energie du niveau fondamental et sa degenerescence

E0 = valeurs_propres[0]
degenerescence = np.sum(np.abs(valeurs_propres - E0) < 1e-10)

print(f"Énergie du fondamental (en unités de J) : {E0}")
print(f"Dégénérescence : {degenerescence}")
print("\nVecteurs propres du fondamental :")

# contenu des vecteurs propres du fondamental

for i in range(len(vecteurs_propres)):
    if np.abs(valeurs_propres[i] - E0) < 1e-10:
        print(vecteurs_propres[i])
```